

WEEK 3, DISCOVER — AUG 26, 2026

Requirements → Product Backlog with AI Agents & Skills

Today your pain notes become a real requirement spec, a backlog, and a working **subagent + skill** — live, on GitHub, before you leave the room.

[Live demo](#) · [real Claude Code](#)

Deliverable: Backlog + spec (draft)

CLO1 Process · CLO3 Tooling

Dr. Prasara Jakkaew, ADT MFU

TODAY • 3 HOURS

What we cover

- 01 Requirements: functional, non-functional, MoSCoW, traceability
- 02 Bridge from W2: your rule.md becomes legal requirements (LR)
- 03 What an AI agent and a Skill actually are
- 04 Live demo: build a requirement-writer agent + audit-backlog skill
- 05 Push it all to GitHub — live, on screen
- 06 Workshop: your company does the same 6 steps, starting now

Law and pain become code an agent can act on

Last week

rule.md

Legal/compliance rules for an AI coding agent — PDPA, CCA §26, ETA.

This week

Spec + backlog + agents

rule.md becomes LR items. An agent writes the spec and keeps the backlog in sync.

Next week

Design, Sep 2

feature-list, user-journey, prototype, architecture + 4 diagrams — built on today's backlog.

Gate W5

Sep 9

Proposal + backlog + design + compliance. No LR in your spec = risk of CONDITIONAL.

Still no production code. DISCOVER ends at the User Validation Gate (W5, Sep 9).

THE ONE SENTENCE

**A good requirement is one
your team **and an AI agent**
understand the same way.**

NOT JUST SOMETHING A HUMAN UNDERSTANDS

TWO KINDS OF REQUIREMENT

What it does vs. how well it does it

FUNCTIONAL

What the system does

- A user can perform a real action
- Written as a user story: As a..., I want..., so that...
- Example: "record a coffee's score + flavor tags"

NON-FUNCTIONAL (NFR)

How well it does it

- Usability, performance, reliability — measurable
- "Fast" is not a requirement. "<2 seconds" is.
- If you can't measure it, you can't test it

User story + INVEST + MoSCoW

As a [user], I want [X], so that [Y] — Independent, Negotiable, Valuable, Estimable, Small, Testable.

#	AS A...	I WANT TO...	SO THAT...	PRIORITY
F1	barista	record a score (1–10) + flavor tags	my notes aren't lost (P1)	Must
F2	barista	see a list of past cuppings	I can compare batches (P2)	Must
F3	barista	pick from a shared flavor-tag list	we score consistently (P3)	Must
F4	owner	add a coffee (name, batch, date)	cupplings attach to the right coffee	Must
F5	barista	filter/sort cupplings by score/coffee	I find the best batch fast	Should
F6	owner	export the history (CSV)	I can share with buyers	Could

Why is F5 a Should, not a Must? It's not the one core workflow (F1) — the thing month-2 BUILD must ship end-to-end.

NFRS THAT ARE ACTUALLY MEASURABLE

Time, count, % — not "fast" or "good"

NFR1 Usability

< 60 seconds

A barista can log one cupping in under 60 seconds.

NFR2 Performance

< 2 seconds

The history list loads in under 2 seconds for 200 cuppings.

NFR3 Reliability

No data loss

No data loss if the phone loses signal mid-save (queue + retry).

An NFR you cannot measure fails the exam. "The system must be fast" is not a requirement.

PRIORITIZATION

MoSCoW — order the backlog, don't just list it

Must

Breaks without it

The core workflow doesn't function. Ships in month 2 (BUILD).

Should

Important, not core

Real value, but the workflow survives without it for now.

Could

Nice if time allows

No harm if it never ships this phase.

Won't

Out of scope

Explicitly not this phase — write it down so no one assumes it's coming.

CupLog's month-2 commitment: B1–B5 (Must only). B6–B7 (Should/Could) only if time allows.

THE RULE THAT GETS CHECKED AT THE GATE

Every requirement traces back to a real pain

Traceability chain pain → requirement → backlog item

INTERVIEW

P1 — "tasting notes on paper/LINE get lost"

REQUIREMENT

F1 — "As a barista, I want to record a score + tags, so that my notes aren't lost"

BACKLOG

B1 — "Record score + flavor tags for a coffee" — **Traces to: F1, P1**

No requirement that's just "made up" with no source. The W5 Gate checks whether your backlog is tied to real interview pain — not just whether a table exists.

Your rule.md becomes LR items in the spec today

Example — from the source workshop's own case legal/PDPA add-on

ASK

"Add a requirement to keep an access log for ≥ 90 days, and get consent for collecting Google Analytics data / IP addresses."

BECOMES LR

LR1 — consent notice + record before storing GA/IP data (PDPA) · **LR2** — access log kept ≥ 90 days (CCA §26)

NOT JUST A FILE

rule.md doesn't sit separately anymore — its Must-have rules turn into LR-numbered items inside today's requirement spec.

Same three laws from W2 (PDPA · CCA §26 · ETA §9/26/28) — now written as LR1, LR2, LR3... inside the spec, not just commands to an agent in a separate file.

One requirement spec file, five parts

.docs/01-requirements/01-spec/20260826-01-cupping-log.md

PROBLEM & USERS

Who's affected + P1–P3, straight from interviews

FUNCTIONAL

F1–F6, user stories with MoSCoW priority (Slide 6)

NON-FUNCTIONAL

NFR1–3, measurable (Slide 7)

LEGAL

LR1–3, pulled from rule.md (Slide 10)

SCOPE

In / out (won't-have) + the ONE core workflow this phase builds

This is the exact file the agent will help write for the rest of this session.

Five core components

1 Perception

Taking in input

Text, files, tool results, environment state — preprocessed before the model reasons about it.

2 Memory

Remembering

In-context (the prompt), external (files/DB), episodic (past actions) — layered for long tasks.

3 Planning

Breaking down the goal

Splits a complex goal into subtasks, handles dependencies between steps.

4 Action

Doing it

Calls tools, directs other subagents, reads/writes files, reports to the user.

5 Self-evaluation

Checking itself

Did the action work? Retry, adjust, continue, or stop — and report progress/errors.

The difference from a chatbot: a chatbot answers once. An agent loops through these five steps until the task is actually done.

A specialist with its own instructions and tools

FILE

.claude/agents/<name>.md

- Required: name, description
- Recommended: tools, model
- Runs in its own context — doesn't see your chat history

 CHANGED SINCE RAISE2'S MATERIAL

No more /agents wizard

- The current Claude Code no longer opens an interactive wizard
- Instead: **type directly to Claude** — "Create a subagent at .claude/agents/... that does X"
- Claude writes the file for you

THREE FILES, THREE JOBS

CLAUDE.md vs. Skill vs. Subagent

FILE	SCOPE	WHEN IT LOADS
CLAUDE.md	Permanent, whole-project rules	Every session, automatically
Skill (SKILL.md)	A specific task/procedure	Only when invoked with /skill-name — saves context
Subagent	A specialist persona + its own tools	Delegated to by Claude, or @-mentioned — separate context

Skill file: `.claude/skills/<name>/SKILL.md` — the invoke command comes from the **folder name**, not the frontmatter.

TODAY'S TARGET — MATCHES CUPLOG \$1 AND THE SOURCE WORKSHOP EXACTLY

What your repo should have by the end of today

Repo tree

```
.claude/  
  agents/  
    requirement-writer.md  
    backlog-auditor.md  
  skills/  
    capture-requirement/  
    audit-backlog/  
.docs/  
  01-requirements/  
    01-spec/{YYYYMMDD}-{no}-{topic}.md  
    backlog.md  
  05-log/{YYYYMMDD}-log.md  
CLAUDE.md  
rule.md          (from W2)
```

Two of these get built live, right now: `requirement-writer` (writes the spec) and `audit-backlog` (checks the backlog is in sync).



Review CLAUDE.md, build the requirement-writer agent

CupLog demo folder already has CLAUDE.md + rule.md from W2

STEP 1

Open the existing CLAUDE.md — no need to re-run `/init`, it's already there

STEP 2 • PROMPT

"Create a subagent at `.claude/agents/requirement-writer.md` — it takes raw pain notes and writes a requirement spec (functional user stories + measurable NFRs + legal rules from `rule.md`) into `.docs/01-requirements/01-spec/{date}-{no}-{topic}.md`, and updates `backlog.md`. If anything is unclear, ask and offer at least 3 options. Never guess."

STEP 3

Show the file Claude actually created — point out `name`, `description`, `tools`, `model` in the frontmatter

We didn't write the syntax ourselves — we described the job in plain language and let Claude write the file. Each company repeats exactly this in the Workshop.



Invoke the agent — get a real spec, live

Prompt

```
Use the requirement-writer agent to write a requirement
spec from this raw pain: baristas' tasting notes on
paper/LINE get lost (P1), hard to compare scores between
batches (P2), new staff score inconsistently (P3).
```

Compare the live output to Slide 11's structure — it should look similar (F1–F4 Must, measurable NFRs, LR citing rule.md). Notice the agent updates `backlog.md` too, not just the spec file — that's the real payoff versus typing it by hand.



Build + invoke the audit-backlog skill

Prompt

```
Create a skill at .claude/skills/audit-backlog/SKILL.md –
it checks that every Must-priority requirement in
.docs/01-requirements/01-spec/ has a matching row in
backlog.md, and every backlog row traces to a real
requirement ID. If unsure whether two items are the same
thing, ask – don't merge them silently.
```

THEN RUN

/audit-backlog — the invoke command comes straight from the folder name

This is agent #2 — backlog-auditor, built here as a Skill instead of a Subagent (either works; a Skill is lighter and invoked directly with /name).



git push — see it land on GitHub

Commands

```
git add .  
git commit -m "W3: requirement-writer agent, audit-backlog skill, spec draft"  
git push
```

Open the real GitHub page so the room sees the files land on the repo. Demo done — every company now does exactly this with their own product, in the Workshop block next.

BEFORE THE WORKSHOP

What you just watched, in 4 steps

Built

requirement-writer agent

Reads raw pain → writes spec + updates backlog.md

Built

audit-backlog skill

Checks spec ↔ backlog stay in sync — invoked with /audit-backlog

Pushed

Everything to GitHub

CLAUDE.md, rule.md, both agents, spec draft, backlog.md

Stuck on git?

See the Appendix

A full git cheat-sheet is at the end of this deck — not part of today's pacing, ask if you need it.

WORKSHOP • START NOW, FINISH BEFORE W4

40 minutes · do this with YOUR product

- 1 Review/extend **your own CLAUDE.md** (already exists from W2)
- 2 Build your own **requirement-writer** subagent — copy the demo, adapt to your product
- 3 Invoke it to write a spec from **your real interview pains**
- 4 Connect your company's **GitHub repo** (create one now if you don't have one) and push
- 5 Invoke the agent to add **LRs from your rule.md** (W2) into the spec
- 6 Build/invoke **audit-backlog**, sync the backlog, commit + push

IN CLASS VS. HOMEWORK

What "good enough for today" looks like

MUST BE ON GITHUB BEFORE YOU LEAVE

- Repo link (created if it didn't exist)
- CLAUDE.md (reviewed/extended)
- requirement-writer.md agent file
- At least one spec draft file

HOMEWORK — BEFORE W4 (SEP 2)

- Add LR from rule.md into the spec
- Build + run audit-backlog, fix any gaps it finds
- Final commit + push of backlog.md

STUCK? COPY, PASTE, SWAP THE DETAILS

Reuse the exact demo prompts

Same 3 prompts as the live demo

- 1) Create a subagent at `.claude/agents/requirement-writer.md`
– it takes raw pain notes and writes a requirement spec (functional user stories + measurable NFRs + legal rules from `rule.md`) into `.docs/01-requirements/01-spec/`, and updates `backlog.md`. If unclear, ask and offer ≥3 options.
- 2) Use the requirement-writer agent to write a spec from this pain: [YOUR real interview pains here].
- 3) Create a skill at `.claude/skills/audit-backlog/SKILL.md`
– checks every Must requirement has a backlog row, and every backlog row traces to a requirement ID. If unsure, ask – don't merge silently. Then run `/audit-backlog`.

WALKING THE ROOM

Two failure patterns to catch

CATCH THIS

Agent instructions with no "ask if unsure" line

- The agent will guess instead of asking
- Fix: add "if anything is unclear, ask and offer ≥ 3 options" to the subagent file

CATCH THIS

Requirements with no traced pain

- A requirement invented on the spot, not from an interview
- Fix: send them back to their interview notes before writing F-items

No GitHub repo yet? Help them create one live now — New repository → private → add every teammate as a collaborator.

BEFORE W4 (SEP 2)

Your actions

01

Finish steps 5–6

LRs into the spec, audit-backlog run and gaps fixed, final commit/push.

02

Keep interviewing

Evidence log still needs to grow toward ≥ 15 real users.

03

Bring to W4

4 diagrams + a prototype sketch — built on top of today's backlog.

Still no production code. DISCOVER ends at the User Validation Gate (W5, Sep 9).

NEXT CLASS

W4 — Design with agents

Sep 2

feature-list, user-journey

The workflows your backlog actually needs, walked step by step.

Sep 2

prototype + 4 diagrams

Architecture, use cases, context, activity — matching CupLog §6.

Everything in W4 is built on top of the backlog and spec you push today — bring the repo link.

NEXT CLASS

See you in Week 4

Sep 2 — Design with agents: feature-list, user-journey, prototype, architecture + 4 diagrams. Bring your Charter, rule.md, and repo link.

Deliverable today: Backlog + spec (draft)

1305493, SE Case Studies, 1/2569

Git cheat-sheet (1/2) — adapted from the source workshop

CATEGORY	COMMANDS
Setup	<pre>git config --global user.name "name" git config --global user.email "email"</pre>
Start a project	<pre>git init (start a new repo) git clone <url> (pull an existing repo down)</pre>
Working with files	<pre>git status git add <file> / git add . git commit -m "message"</pre>
History	<pre>git log git diff (see changes before you add them)</pre>

Source: adapted from RAISE2_W1D2_Workshop.pdf's git cheat-sheet infographic, restyled to match this deck's design — command content matches the original.

Git cheat-sheet (2/2) — remote, branches, undo

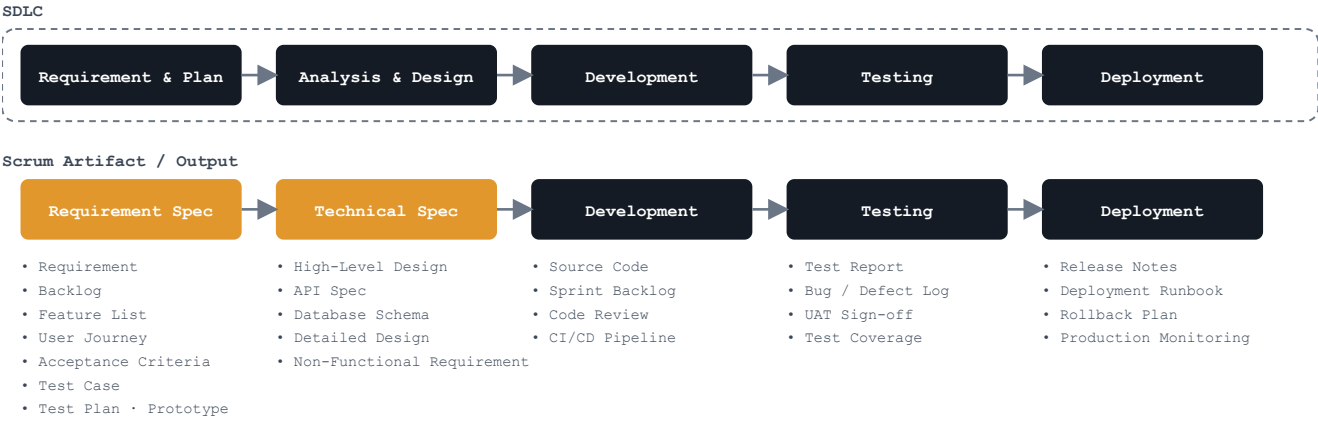
CATEGORY	COMMANDS
Remote (GitHub)	<code>git remote add origin <url></code> <code>git push</code> <code>git pull</code>
Branching	<code>git branch</code> (list branches) <code>git checkout -b <name></code> (create + switch)
Undo / restore	<code>git restore <file></code> (discard unstaged edits) <code>git reset</code> (unstage, keeps content)

edit → add → commit → push

- 1 **Edit** — change a file locally (spec, backlog, an agent file)
- 2 **git add .** — stage what changed
- 3 **git commit -m "..."** — record it with a message
- 4 **git push** — send it to GitHub so the team and the grader can see it

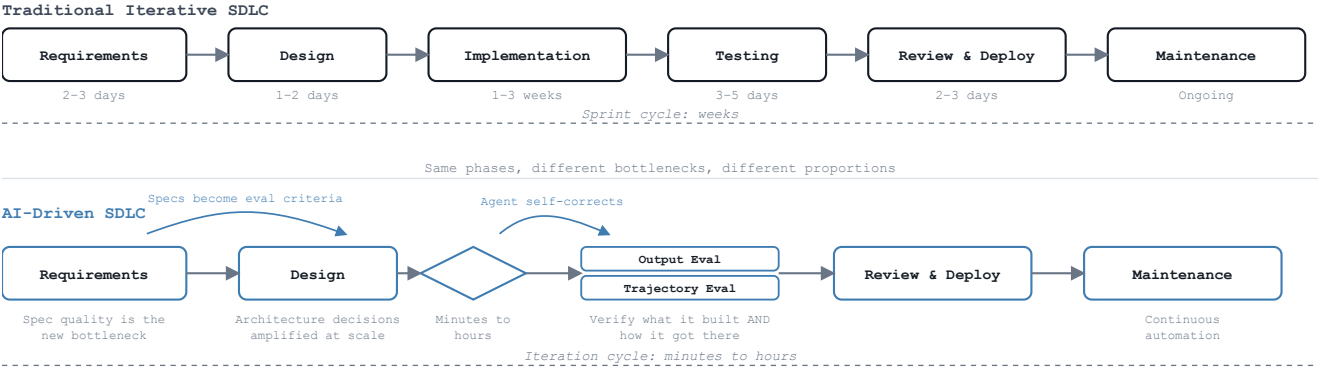
Do this every time you or an agent changes a file under `.claude/` or `.docs/` — a spec that only exists on one laptop doesn't count at the Gate.

SDLC and Agile with AI



Everything you built today (backlog, spec) and its bullets under **Requirement Spec** — Week 4's design work becomes the **Technical Spec** bullets next to it. Same pipeline, one step at a time.

New SDLC with Vibe Coding



11 agents — meet the team

AGENT	OWNS	ROLE
<code>requirement-writer</code>	01-spec/ + backlog	Raw requirements → spec (today's agent)
<code>backlog-auditor</code>	backlog.md	Spec ↔ backlog consistency (today's skill)
<code>feature-journey-writer</code>	feature-list + user-journey	Backlog → features + MoSCoW + Mermaid
<code>architecture-writer</code>	architecture.md	High-level design
<code>api-db-writer</code>	api-spec + db-spec	API contract + ER model, together
<code>detailed-design-writer</code>	detailed-design/	Per-feature component design
<code>nfr-reviewer</code>	nfr-review.md	Verify NFRs are designed for — read-only
<code>test-writer</code>	03-testing/	Acceptance + test-plan + test-cases
<code>prototype-writer</code>	01-prototypes/	Build HTML mockups
<code>prototype-auditor</code>	—	Flag a stale prototype — read-only
<code>phase-planner</code>	02-plan + 03-task	Split phases + break down tasks

Today you only needed `requirement-writer` + `backlog-auditor` (rows 1–2). The rest exist because the pipeline keeps going past requirements — into design, testing, and planning.

14 skills, grouped by what they do

Create work

```
/capture-requirement  
/build-prototype
```

Sync one layer

```
/audit-backlog  
/sync-feature-journey  
/sync-architecture  
/sync-api-db  
/sync-detailed-design  
/sync-test-plan  
/sync-phase-plan
```

Combined phases

```
/run-requirements-phase  
/run-technical-phase  
/run-prototype-phase  
/sync-technical-spec
```

Audit the chain

```
/audit-pipeline  
spec → endpoints, one command
```

Key rule this project follows: when a task matches a skill, **call that skill instead of editing files directly** — it preserves cross-file consistency checks and automatic log recording, the same reason today's `audit-backlog` exists.